

第二单元：Webpack

一、Webpack

1. 核心概念

1.1 什么是Webpack?

1.2 为什么需要Webpack?

2. 核心概念

2.1 五大核心要素

2.2 依赖图 (Dependency Graph)

3. 安装与基本配置

3.1 环境准备

3.2 基本配置文件

3.3 打包脚本配置

4. 处理不同类型资源

4.1 处理CSS资源

4.2 处理预处理器 (Less/Sass)

4.3 处理图片和字体资源

4.4 处理ES6+语法 (Babel)

5. 常用插件配置

5.1 HtmlWebpackPlugin

5.2 提取CSS文件

6. 开发环境配置

6.1 Webpack Dev Server

6.2 Source Map配置

6.3 热模块替换 (HMR)

7. 多入口与多出口配置

7.1. 基本配置

7.2. 为每个页面生成对应的 HTML

8. 配置路径别名 (Resolve Alias)

9. 开发服务器代理 (Proxy) 配置

10. 高级特性与优化

10.1 代码分割

10.2 环境分离

10.3 缓存优化

二、ES Module：模块化规范

1. 导出 (Export)

2. 导入 (Import)

3. ES Module 与 CommonJS 的区别

一、Webpack

1. 核心概念

1.1 什么是Webpack?

Webpack 是一个**静态模块打包工具**，用于处理现代 JavaScript 应用程序。当 Webpack 处理应用程序时，它会在内部构建一个**依赖图**（dependency graph），此依赖图对应映射到项目所需的每个模块，并生成一个或多个bundle。

核心定位：webpack是一个静态的模块化打包工具。

1.2 为什么需要Webpack?

在前端开发复杂化的今天，我们面临诸多挑战：

- 需要通过**模块化**的方式来开发
- 使用ES6+、TypeScript等高级特性
- 使用Sass、Less等CSS预处理器
- 需要**实时监听文件变化**提高开发效率
- 代码需要**压缩、合并及其他优化**

Webpack正是为了解决这些问题而诞生。

2. 核心概念

2.1 五大核心要素

概念	说明	作用
Entry（入口）	指示webpack从哪个文件开始打包	构建依赖图的起点
Output（输出）	指示webpack打包好的文件输出位置和命名	指定打包后文件的存放位置
Loader（加载器）	处理非JS文件（如CSS、图片等）	将其他资源转换为webpack能处理的模块
Plugins（插件）	扩展webpack功能	执行更广泛的任务如代码压缩、文件复制等
Mode（模式）	开发模式或生产模式	启用相应模式下的内置优化

2.2 依赖图（Dependency Graph）

Webpack从入口文件开始，递归地构建一个包含应用程序所有模块的依赖关系图，然后根据这个图来打包所有模块。

3. 安装与基本配置

3.1 环境准备

首先确保已安装Node.js，然后初始化项目：

▼

Bash

```
1  # 创建项目目录并初始化
2  npm init -y
3
4  # 局部安装webpack（推荐）
5  npm install webpack webpack-cli --save-dev
```

推荐局部安装，因为不同项目可能依赖不同版本的webpack，这样可以避免全局版本冲突的问题。

3.2 基本配置文件

创建 `webpack.config.js` 配置文件:

```
1  const path = require('path');
2
3  module.exports = {
4    // 入口文件
5    entry: './src/index.js',
6
7    // 输出配置
8    output: {
9      filename: 'bundle.js',
10     path: path.resolve(__dirname, 'dist'), // 必须为绝对路径
11     clean: true, // webpack5特性, 打包前清空输出目录
12   },
13
14   // 模式配置
15   mode: 'development', // 开发模式, 生产模式用 'production'
16 };
```

3.3 打包脚本配置

在 `package.json` 中添加打包命令:

```
1  {
2    "scripts": {
3      "build": "webpack"
4    }
5  }
```

执行 `npm run build` 即可进行打包。

4. 处理不同类型资源

4.1 处理CSS资源

安装并配置CSS相关的loader:

```
1 npm install --save-dev css-loader style-loader
```

webpack.config.js配置:

```
1 module.exports = {  
2   module: {  
3     rules: [  
4       {  
5         test: /\.css$/i,  
6         use: [  
7           'style-loader', // 将JS中的CSS通过style标签添加到HTML  
8           'css-loader'    // 将CSS编译成CommonJS模块  
9         ],  
10        // loader的执行顺序是从后往前的[4](@ref)  
11      }  
12    ]  
13  }  
14  };
```

4.2 处理预处理器 (Less/Sass)

```
1  
2 # 处理Less  
3 npm install less less-loader --save-dev  
4  
5 # 处理Sass/Scss  
6 npm install sass sass-loader --save-dev
```

配置示例:

```
1
2 {
3   test: /\.less$/i,
4   use: [
5     'style-loader',
6     'css-loader',
7     'less-loader', // 将Less编译为CSS文件
8   ],
9 },
10 {
11   test: /\.s[ac]ss$/i,
12   use: [
13     'style-loader',
14     'css-loader',
15     'sass-loader', // 将Sass编译成CSS
16   ],
17 }
```

4.3 处理图片和字体资源

Webpack 5可以使用内置的Asset Modules:

```
1
2
3 {
4   test: /\. (png|jpg|jpeg|gif|svg)$/ ,
5   type: 'asset',
6   parser: {
7     dataUrlCondition: {
8       maxSize: 4 * 1024, // 小于4kb转换为base64
9     }
10  },
11  generator: {
12    filename: 'images/[name]-[hash:7][ext]' // 输出路径和文件名
13  }
14 }
```

4.4 处理ES6+语法 (Babel)

安装Babel相关依赖：

```
▼ Bash |
1
2  npm install --save-dev babel-loader @babel/core @babel/preset-env
```

配置规则

:

```
▼ JavaScript |
1
2
3  {
4    test: /\.js$/,
5    exclude: /node_modules/, // 排除node_modules
6    use: {
7      loader: 'babel-loader',
8      options: {
9        presets: ['@babel/preset-env']
10     }
11  }
12 }
```

也可以使用 `.babelrc` 文件单独配置Babel。

5. 常用插件配置

5.1 HtmlWebpackPlugin

自动生成HTML文件并注入打包后的资源

:

```
1  const HtmlWebpackPlugin = require('html-webpack-plugin');
2
3  module.exports = {
4    plugins: [
5      new HtmlWebpackPlugin({
6        template: './public/index.html', // 模板文件
7        filename: 'index.html', // 输出文件名
8        inject: 'body' // 脚本注入位置
9      })
10   ]
11  };
```

5.2 提取CSS文件

使用MiniCssExtractPlugin将CSS提取到单独文件：

```
1  const MiniCssExtractPlugin = require('mini-css-extract-plugin');
2
3  module.exports = {
4    module: {
5      rules: [
6        {
7          test: /\.css$/i,
8          use: [MiniCssExtractPlugin.loader, 'css-loader']
9        }
10      ]
11    },
12    plugins: [
13      new MiniCssExtractPlugin({
14        filename: 'styles/chunk-[contenthash].css'
15      })
16    ]
17  };
```

6. 开发环境配置

6.1 Webpack Dev Server

提供实时重新加载的开发服务器：

```
▼ Bash |
1 npm install --save-dev webpack-dev-server
```

配置package.json脚本：

```
▼ JSON |
1 {
2   "scripts": {
3     "dev": "webpack serve --open"
4   }
5 }
```

webpack.config.js配置：

```
▼ JavaScript |
1 module.exports = {
2   devServer: {
3     static: './dist',
4     port: 3000,
5     hot: true, // 热模块替换
6     compress: true, // 压缩
7     open: true // 自动打开浏览器
8   }
9 };
```

6.2 Source Map配置

便于调试，定位原始代码错误位置：

```
▼ JavaScript |
1 module.exports = {
2   devtool: 'cheap-module-eval-source-map' // 开发环境推荐
3   // 生产环境可用 'source-map' 或省略
4 };
```

6.3 热模块替换（HMR）

实现局部模块更新而不刷新页面：

```
1
2 devServer: {
3   hot: true // 开启热更新
4 }
```

7. 多入口与多出口配置

多入口配置主要用于多页面应用（MPA），每个入口文件会对应生成一个打包后的 bundle。

7.1. 基本配置

在 `entry` 中设置多个入口点，在 `output` 中使用 `[name]` 占位符来生成对应的输出文件。

```
1 // webpack.config.js
2 const path = require('path');
3
4 module.exports = {
5   // 1. 定义多个入口
6   entry: {
7     home: './src/pages/home/index.js',
8     about: './src/pages/about/index.js',
9     contact: './src/pages/contact/index.js'
10  },
11   // 2. 配置多出口，使用 [name] 占位符
12   output: {
13     filename: 'js/[name].[contenthash:8].bundle.js',
14     path: path.resolve(__dirname, 'dist'),
15     clean: true // 每次构建前清理输出目录
16   }
17 };
```

7.2. 为每个页面生成对应的 HTML

使用 `HtmlWebpackPlugin` 为每个入口生成对应的 HTML 文件，并通过 `chunks` 指定需要引入的 JavaScript 文件

```
1
2  const HtmlWebpackPlugin = require('html-webpack-plugin');
3
4  module.exports = {
5    // ... 其他配置
6    plugins: [
7      // 首页
8      new HtmlWebpackPlugin({
9        template: './src/pages/home/index.html',
10       filename: 'home.html',
11       chunks: ['home'] // 只包含 home 入口的 bundle
12     }),
13     // 关于我们页
14     new HtmlWebpackPlugin({
15       template: './src/pages/about/index.html',
16       filename: 'about.html',
17       chunks: ['about'] // 只包含 about 入口的 bundle
18     }),
19     // 联系我们页
20     new HtmlWebpackPlugin({
21       template: './src/pages/contact/index.html',
22       filename: 'contact.html',
23       chunks: ['contact'] // 只包含 contact 入口的 bundle
24     })
25   ]
26 };
```

8. 配置路径别名 (Resolve Alias)

路径别名可以简化模块引入路径，提高代码可读性和维护性。

```
1 // webpack.config.js
2 const path = require('path');
3
4 module.exports = {
5   // ... 其他配置
6   resolve: {
7     alias: {
8       '@': path.resolve(__dirname, 'src/'),
9       '@components': path.resolve(__dirname, 'src/components/'),
10      '@utils': path.resolve(__dirname, 'src/utils/'),
11      '@assets': path.resolve(__dirname, 'src/assets/'),
12      '@styles': path.resolve(__dirname, 'src/styles/')
13    },
14    // 自动解析扩展名, 导入文件时可以省略后缀
15    extensions: ['.js', '.jsx', '.json', '.css', '.scss']
16  }
17 };
```

配置后, 在代码中可以这样使用:

```
1
2 // 配置前
3 import Button from '../../components/Button';
4
5 // 配置后
6 import Button from '@components/Button';
7 import utils from '@utils/helpers';
```

9. 开发服务器代理 (Proxy) 配置

Webpack DevServer 的代理功能主要用于解决开发阶段的跨域问题, 其底层基于 `http-proxy-middleware`

```
1 module.exports = {
2   devServer: {
3     proxy: [
4       {
5         context: ['/api'], // 代理以 /api 开头的请求
6         changeOrigin: true, // 修改请求头中的 Origin 为目标 URL
7         target: 'http://localhost:3000', // 后端服务器地址
8         pathRewrite: { '^/api': '' }, // 将 /api 前缀替换为空字符串
9         // 例如 /api/user 会被代理到 http://localhost:3000/user
10      }
11    ]
12  }
13 };
```

10. 高级特性与优化

10.1 代码分割

通过入口点和SplitChunksPlugin进行代码分割：

```
1 module.exports = {
2   entry: {
3     index: './src/index.js',
4     another: './src/another.js'
5   },
6   output: {
7     filename: '[name].bundle.js', // 使用[name]占位符
8     path: path.resolve(__dirname, 'dist')
9   },
10  optimization: {
11    splitChunks: {
12      chunks: 'all' // 分离公共依赖
13    }
14  }
15 };
```

10.2 环境分离

分离开发和生产环境配置：

1. 创建通用配置文件 `webpack.common.js`
2. 创建开发环境配置 `webpack.dev.js`
3. 创建生产环境配置 `webpack.prod.js`

使用webpack-merge合并配置：

```
1 // webpack.prod.js
2 const { merge } = require('webpack-merge');
3 const common = require('./webpack.common.js');
4
5 module.exports = merge(common, {
6   mode: 'production',
7   devtool: 'source-map',
8   // 生产环境特定配置
9 });
```

10.3 缓存优化

使用内容哈希命名实现缓存优化：

```
1
2 output: {
3   filename: 'js/[name]-[contenthash].js',
4   chunkFilename: 'js/[name]-[contenthash].chunk.js'
5 },
6 plugins: [
7   new MiniCssExtractPlugin({
8     filename: 'css/[name]-[contenthash].css'
9   })
10 ]
```

二、ES Module：模块化规范

1. 导出 (Export)

- **命名导出 (Named Exports)** : 可以导出多个值。导入时需要用相同的名称。

JavaScript

```
1 // 导出单个值
2 export const name = 'value';
3
4 // 导出多个值
5 export const name1 = 'value1';
6 export const name2 = 'value2';
7
8 // 导出函数
9 export function functionName() {}
10
11 // 导出类
12 export class ClassName {}
```

- **默认导出 (Default Exports)** : 每个模块只能有一个默认导出。导入时可以使用任意名称。

JavaScript

```
1
2 // 导出一个值作为默认导出
3 const name = 'value';
4 export default name;
5
6 // 直接导出字面量、函数或类
7 export default 'value';
8 export default function () {}
9 export default class {}
```

2. 导入 (Import)

- **导入命名导出**: 导入时需与导出时的名称相匹配。

JavaScript

```
1 import { name1, name2 } from './module.js';
2
3 // 重命名导入的特性
4 import { name1 as newName1 } from './module.js';
```

- **导入默认导出**:

```
1 import anyName from './module.js';
```

- 导入命名导出和默认导出：

```
1 import defaultExport, { name1, name2 } from './module.js';
```

- 整体导入（Namespace import）：将模块中的所有导出作为一个对象导入。

```
1 import * as moduleName from './module.js';
```

- 导入副作用（仅执行模块中的代码，不导入任何值）：

```
1 import './module.js';
```

3. ES Module 与 CommonJS 的区别

特性	ES Module	CommonJS
加载方式	静态（编译时）	动态（运行时）
导入语法	<code>import</code>	<code>require()</code>
导出语法	<code>export</code>	<code>module.exports</code>
引用类型	实时绑定	值拷贝
加载时机	异步	同步
顶层 <code>this</code>	<code>undefined</code>	模块导出对象